Core Image Representation and Matrix Operations

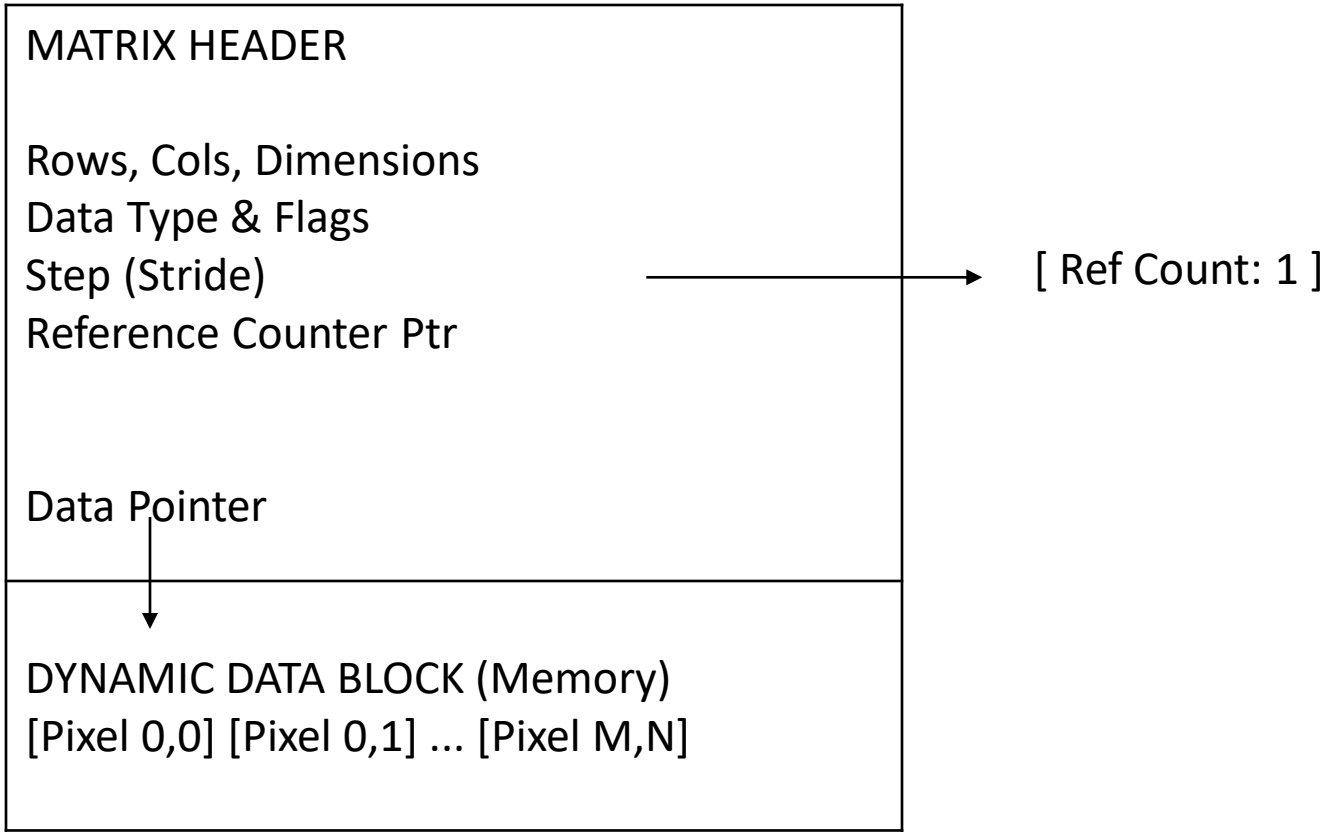
MAT Structure

- In OpenCV, the cv::Mat class is the foundational data structure used to store and manipulate images and multi-dimensional matrices.
- It is divided into two primary sections:

➢ The Header: containing metadata like dimensions, channels, and data type and

➢ The Data Block: holding the actual pixel values.

cv::Mat Object



Header (Metadata)

The header contains critical information about the matrix structure:

- **rows and cols:** The height and width of the image or matrix dimensions.
- **dims:** The number of dimensions e.g., 2 for standard images, 3 for volumetric data.
  - **dims = 1**: A 1D array (a flat list of numbers). Examples include a audio signal or a time-series line.
  - **dims = 2**: A 2D array (rows and columns). Examples include a grayscale image (height × width) or a spreadsheet
  - **dims = 3**: A 3D array (height × width × depth). Examples include a color image (RGB channels count as the 3rd dimension) or medical MRI/CT scans.
  - **dims = 4**: A 4D array. Examples include a video (frames × height × width × channels) or a batch of color images used in machine learning training.

- **channels:** The number of data values per pixel. For example, a grayscale image has 1 channel, while a standard RGB image has 3 channels.
- **depth and type:** Specifies the bit-depth and data type

  (e.g., CV_8U for an 8-bit unsigned integer,

  CV_32F for 32-bit float)

  combined with the number of channels

  (e.g., CV_8UC3 for an 8-bit RGB image).

- step: An array of byte strides (M.step[])
- Byte Stride (step[0]):

  The total number of bytes physically required to span from the start of one row to the start of the next row.
- Column Stride (step[1]):

  The number of bytes required to move from one pixel/element to the next within the same row.

- This strided approach is particularly common in image processing libraries like OpenCV.
- When navigating an image where the top-left corner starts at memory pointer data, finding the memory address of a specific pixel at coordinates (y, x) requires this offset calculation:
- $$\text{Address} = \text{data} + (y \times \text{step}[0]) + (x \times \text{step}[1])$$

  -data : pointer to the very first byte

  -step[0]: no.of bytes a full row occupies in memory (including padding)

  -step[1]: no.of bytes a single element (pixel) occupies.

- Padding: Many hardware architectures perform faster when data addresses are aligned to specific boundaries (like 4, 8, or 16 bytes). step[0] accounts for the extra padding bits inserted at the end of rows.
- Sub-matrices (ROI): When dealing with a cropped sub-image, step[0] allows code to easily "jump" the right amount of bytes to access the next line.

Management & Reference Counting

Understanding how cv::Mat handles memory is vital for computer vision development. cv::Mat manages this block by utilizing smart pointer logic to optimize performance

- **Shallow Copy by Default**: Assigning one matrix to another (e.g., Mat B = A;) or passing a matrix to a function copies only the header. Both headers then point to the identical data block, and the internal reference counter increments.
- **Automatic Deallocation**: When a cv::Mat object leaves scope, its destructor decrements the reference counter. If the counter hits zero, the underlying data block is safely freed from system memory.
- **Deep Copying**: To replicate the underlying pixel contents into a completely independent block of memory, you must explicitly use the .clone() or .copyTo() methods.

Data Layout in Memory

- By default, the elements in a cv::Mat array are stored in row-major order. This means rows are arranged sequentially, one after another in a contiguous or near-contiguous block of system memory.
- **Interleaved Channels**: For multi-channel arrays (such as BGR color images), channel values belonging to a single pixel are interleaved directly next to each other in memory. For example, the byte pattern for a 3-channel BGR pixel structure looks like:

  B0 G0 R0 B1 G1 R1 B2 G2 R2....
- **Addressing Formula**: To calculate the exact memory address of any element at coordinates (y, x) in a 2D array, OpenCV computes $\text{Address} = \text{data} + (y \times \text{step}[0]) + (x \times \text{step}[1])$

- In Python, OpenCV seamlessly integrates with the NumPy library, making Mat objects essentially represented as multi-dimensional NumPy arrays.